

Day 12: Kubernetes Architecture Deep Dive

Control Plane Services, Worker Node agents, Core Objects & Deployment Spec

Module 1: Kubernetes Core Architecture Deep Dive

Kubernetes organizes container workloads into clustered node groups. The control plane manages the administrative lifecycle of these objects, ensuring the current running state of the cluster constantly matches the user-defined desired configuration state (defined in YAML files).

Detailed Control Plane Subsystems

- **Kube-APIserver:** The primary communication hub. Validates and configures data for the API objects, including pods, services, replication controllers, and deployments. It handles both external user queries (kubectl) and internal cluster notifications.
- **etcd Key-Value Store:** Stores all configuration details and runtime cluster states. Because etcd is highly-available and consistent, write accesses must go through the API Server to avoid corruption.
- **Kube-Scheduler:** Evaluates hardware specifications, affinity rules, data locality requirements, and network policies to assign Pods to optimal Node hosts.
- **Kube-Controller-Manager:** Compiles all logical controllers. It acts as a continuous loop comparing the actual cluster state (e.g. 2 replicas running) against the desired state (e.g. 3 replicas requested) and executes actions to resolve differences.

Worker Node Agent Subsystems

- **Kubelet:** The node host commander. Monitors the API Server for Pods assigned to its host, interacts with the local container runtime to launch container packages, and performs health checks (liveness/readiness probes).
- **Kube-Proxy:** An essential networking daemon. Programs host IPTables or IPVS configurations to route network traffic sent to Kubernetes Services directly to the target Pod backends.
- **Container Runtime Interface (CRI):** Abstraction layer allowing Kubernetes to use various container runtimes (e.g. containerd, CRI-O, Docker Engine) to pull images and execute containers.

Module 2: Core Kubernetes Objects & Lifecycle

Kubernetes features persistent entities that represent the state of the cluster. Once created, Kubernetes works continuously to ensure those objects exist.

Primary Object Types & Scopes

Object Kind	Operational Definition	Typical Use Case
Pod	Smallest deployable execution unit containing one or more containers	Running a single Flask application microservice container
Deployment	Declarative controller managing replica sets and updates	Deploying web servers with rolling updates and zero downtime
ReplicaSet	Low-level controller ensuring a specified number of Pods run	Maintained automatically by Deployments to handle Pod failures
Service	Logical abstraction providing stable IP and DNS entry to Pods	Load balancing requests across dynamic, ephemeral backend Pods
Namespace	Logical partition of cluster resources for multi-tenancy	Separating Development, Staging, and Production environments

Service Types Comparison

- **ClusterIP (Default):** Exposes the service on a cluster-internal IP address. The service is only reachable from within the cluster. Used for internal backend database links.
- **NodePort:** Exposes the service on each Node's IP at a static port (usually 30000-32767). Allows external traffic to reach the cluster directly via <NodeIP>:<NodePort>.
- **LoadBalancer:** Exposes the service externally using a cloud provider's external load balancer (e.g., AWS NLB/ALB). Automatically routes traffic down to NodePorts and Pods.

Module 3: Declarative Configuration & Deployment Blueprints

Kubernetes workloads are defined declaratively in YAML configuration sheets. These manifests declare the desired end-state, allowing K8s controllers to build it automatically. Below is the blueprint of a standard deployment configuration hosting the CarePlus Hospital application, incorporating health check labels and replicas.

Application Deployment Specification: deployment.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: careplus-deployment
  labels:
    app: careplus
spec:
  replicas: 3
  selector:
    matchLabels:
      app: careplus
  template:
    metadata:
      labels:
        app: careplus
    spec:
      containers:
        - name: careplus-container
          image: ubuntu-hospital-web:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 5000
```

Deployment Network Interface: service.yml

```
apiVersion: v1
kind: Service
metadata:
  name: careplus-service
spec:
  type: NodePort
  selector:
    app: careplus
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
      nodePort: 32000
```

In the configurations above, the Deployment runs 3 replicas of the CarePlus container. The Service exposes the application through a NodePort interface. Any requests sent to port 32000 on the Kubernetes

host are automatically load-balanced across the 3 backend CarePlus containers running on port 5000.